

TRIVERIFY

3-Layer Legal Metrology Compliance Scanner

BUILD MANUAL · V1

Purpose: convert the supplied SIH26034 problem statement, the TriVerify solution deck, and the supplied CHIRAAG Build Manual into one executable engineering plan for a six-person hackathon team.

Stack	Python · FastAPI · React · PostgreSQL
Core	Computer Vision + OCR + NLP normalization + deterministic rules
Differentiator	Photo ↔ e-commerce listing ↔ government record triangulation
Primary output	Explainable compliance classification + evidence-backed report
Build philosophy	Contract first · vertical slice early · fixtures · offline demo · feature freeze

SOURCE BASIS

This manual preserves the supplied source framing. The SIH analysis says PS034 asks for automated extraction/validation and rule-based compliance checking; the TriVerify deck proposes the three-layer cross-verification concept. The CHIRAAG manual is used as the engineering structure: frozen contracts, explicit modules, six-person ownership, early vertical slice, committed demo data, fallbacks, validation, and a rehearsed offline-first demo.

1. ORIENTATION

What we are actually building

TriVerify is an evidence-backed compliance assistant that takes a packaged-product image, extracts the legally relevant declarations, compares them with the corresponding e-commerce listing and the registered government declaration, and produces a traceable classification with the exact mismatches and visual evidence.

The supplied SIH26034 analysis identifies mandatory declarations including manufacturer/packer/importer details, net quantity, MRP, date information and consumer-care details, along with correctness/completeness/placement, readability and font-size checks. The proposed solution therefore keeps AI/OCR on the extraction side and keeps the final compliance decision deterministic and rule-based.

The TriVerify deck adds the central three-layer idea: a seller can manipulate one surface while leaving another inconsistent. Pairwise comparison across the uploaded photo, live listing, and official record makes the discrepancy visible instead of relying on a single source.

North-star demo sentence

"Show me what is printed on the pack, what the seller claims online, what the official record says, and exactly why those three do or do not agree."

Non-negotiable engineering principles

- **Deterministic compliance:** AI extracts and normalizes; explicit rules decide compliance.
- **Evidence over claims:** every important mismatch should point to a source field and, where possible, an image crop.
- **Unknown is not compliant:** missing or low-confidence evidence becomes an explicit review state, not a silent guess.
- **Contract first:** freeze database entities and API JSON before parallel implementation.
- **Vertical slice early:** one product must travel end-to-end before the team expands scope.
- **Offline-first demo:** cached fixtures, source snapshots and reports must keep the demo alive if external services fail.
- **Measure uncertainty:** OCR confidence, source availability and rule confidence should be visible.
- **Cut scope aggressively:** a tight, explainable four-feature demo beats seven half-working features.

2. REQUIREMENTS → IMPLEMENTATION MAP

Source requirements

Requirement from supplied PS/deck	TriVerify implementation
Scan products, images and labels	Web upload + image preprocessing + OCR/layout extraction
Detect mandatory declarations	Structured field extractor for MRP, quantity, dates, manufacturer/packer/importer, consumer-care
Check correctness and completeness	Field-level rule checks against source records
Placement/readability/font-size checks	Image-region checks; mark unsupported/low-confidence checks for human review
Cross-check online listing	E-commerce adapter + normalized listing schema
Cross-check government record	e-Maap/CLMS adapter + normalized official declaration schema
Generate violation reports	Evidence-backed report with rule IDs, mismatches, confidence and source references
Inspection history/dashboard	Minimal officer dashboard with searchable case list and case detail
PDF/editable export	Generate a case report from the final structured result
Role-based access	Keep as post-MVP unless required by judging/demo; demo can use one officer role

Hackathon interpretation

The supplied material supports a web/mobile compliance application and a deterministic rules engine. It does **not** require a large autonomous AI platform. Therefore the MVP should prove the complete evidence chain rather than attempting a production-grade marketplace crawler, full legal-rule universe, or large model-training pipeline.

MVP acceptance test

- 1 Upload one product image.
- 2 Extract the key declarations and show their source regions.
- 3 Load the corresponding seller/e-commerce record.
- 4 Load the corresponding official declaration.
- 5 Normalize comparable fields into common units/formats.
- 6 Run deterministic pairwise and three-way rules.
- 7 Classify the case as Compliant, Seller Fraud, Counterfeit/Illegal Import, or Manual Review where evidence is insufficient.
- 8 Show the mismatch evidence and generate a report.

3. SYSTEM ARCHITECTURE

Two planes: ingestion/analysis and live review

Following the CHIRAAG pattern, keep data acquisition and heavy preprocessing separate from the demo service. The offline/fixture pipeline produces normalized case data; the online service serves those cases, runs rules, and renders evidence. This prevents the live demo from depending on a fragile external API.



Layer	Responsibility	Demo dependency
Image pipeline	Preprocess → detect text regions → OCR → field extraction	Cached images + local model/fixture
Source adapters	Fetch/parse e-commerce and government records	Prefer cached snapshots for demo
Normalization	MRP, quantity units, dates, names, identifiers	Local deterministic code
Verification engine	Pairwise + three-way comparisons and rule evaluation	Fully local
Evidence store	Crops, raw snippets, source timestamps, rule references	Fully local
API	Case creation, case result, evidence retrieval, report generation	Fully local
Frontend	Officer workflow, comparison table, evidence viewer, report	Fully local

Recommended deployment

- Docker Compose for PostgreSQL and backend services.
- React frontend served locally during the demo.
- A seeded demo database containing the complete end-to-end cases.
- A local **fixtures/** directory containing representative OCR output, e-commerce responses, government responses, and image crops.
- External integrations can be enabled when credentials/network are available, but the demo path must not require them.

4. DATA MODEL — FREEZE AT H0

The following schema is a proposed TriVerify contract derived from the source requirements. It deliberately keeps the model small enough for a hackathon while preserving provenance and explainability.

Entity	Key fields	Purpose
cases	case_id, created_at, status, classification, overall_confidence	One inspection / verification case
products	product_id, case_id, name, brand, category, identifier	Canonical product identity
image_observations	obs_id, product_id, image_path, field_name, value_raw, value_norm, bbox, confidence	OCR/CV extraction with visual provenance
listing_snapshots	snapshot_id, product_id, source, url_or_id, captured_at, raw_json	Seller/e-commerce evidence snapshot
listing_fields	snapshot_id, field_name, value_raw, value_norm, confidence	Normalized seller claims
official_records	record_id, product_id, source, captured_at, raw_json	e-Maap/CLMS evidence snapshot
official_fields	record_id, field_name, value_raw, value_norm	Normalized official declarations
rules	rule_id, field_name, rule_type, severity, explanation_template, active	Deterministic compliance rules
rule_results	result_id, case_id, rule_id, status, severity, explanation, evidence_refs	Auditable decision output
evidence	evidence_id, case_id, source_type, source_ref, crop_path, excerpt, metadata	Evidence attached to a finding
reports	report_id, case_id, generated_at, file_path	Generated PDF/editable report

Canonical field vocabulary

mrp · net_quantity · quantity_unit · manufacturing_date · packing_date · expiry_date · manufacturer_name · packer_name · importer_name · consumer_care · product_name · brand · identifier.

Contract rule

Once the schema and API response shape are committed, changes require team agreement. Individual modules may evolve internally, but their boundary contracts do not.

5. API CONTRACT

Keep the backend intentionally small. The CHIRAAG manual recommends a few stable endpoints rather than a sprawling API. TriVerify should follow the same principle.

Method	Endpoint	Purpose
POST	/cases	Create a verification case from an uploaded image + product identifier/source hints
GET	/cases/{case_id}	Return complete verification result, field comparisons, findings and confidence
GET	/cases/{case_id}/evidence/{evidence_id}	Serve image crop / source excerpt / provenance
POST	/cases/{case_id}/verify	Run or re-run deterministic verification
GET	/cases	Minimal inspection history/search
GET	/cases/{case_id}/report	Generate/serve the evidence-backed report
GET	/health	Health check for demo/debugging

Frozen result shape

```
{ "case_id": "TV-0001", "classification": "SELLER_FRAUD", "overall_confidence": 0.94, "fields": {
  "mrp": { "photo": {"raw": "■99", "norm": 99}, "listing": {"raw": "■79", "norm": 79}, "official":
{"raw": "■99", "norm": 99}, "status": "MISMATCH" }, "net_quantity": { "photo": {"raw": "500 g", "norm":
{"value": 500, "unit": "g"}}, "listing": {"raw": "500 g", "norm": {"value": 500, "unit": "g"}},
"official": {"raw": "500 g", "norm": {"value": 500, "unit": "g"}}, "status": "MATCH" } }, "findings": [
{ "rule_id": "MRP-001", "severity": "HIGH", "status": "FAIL", "explanation": "Photo and official record
agree while listing differs.", "evidence_ids": ["EV-11", "EV-12", "EV-13"] } ] }
```

Important: the exact rule IDs and final legal wording must be reviewed against the applicable source material before claiming a legal violation. The engineering system should explain what it observed and which configured rule fired, rather than inventing legal conclusions.

6. MODULE BREAKDOWN

Every module has JOB → INPUT → OUTPUT → DONE WHEN → ALGORITHM → CONSTRAINTS. This is intentionally close to the CHIRAAG build-manual structure.

M	Module	Owner	Priority
1	Image ingestion + preprocessing	Data/CV	P0
2	OCR + declaration extraction	CV/NLP	P0
3	Source adapters / snapshots	Backend/Data	P0
4	Field normalization	NLP/Rules	P0
5	Deterministic compliance engine	Rules	P0
6	Triangulation / classification	Rules + Lead	P0
7	Evidence + report generation	Backend	P0
8	Officer dashboard	Frontend	P0
9	History/search + polish	Frontend/Backend	P1

M1 — Image ingestion + preprocessing

JOB: turn a product photo into stable input for OCR. **INPUT:** JPG/PNG. **OUTPUT:** corrected image, detected text regions, quality score. **DONE WHEN:** the same demo image produces repeatable regions and a visible preprocessing preview. **ALGORITHM:** resize, denoise, contrast/illumination correction, perspective handling where practical, region detection. **CONSTRAINT:** never destroy the original; store both original and derived image.

M2 — OCR + declaration extraction

JOB: extract MRP, quantity, dates and manufacturer/packer/importer/consumer-care fields. **INPUT:** preprocessed image. **OUTPUT:** field/value/bbox/confidence tuples. **DONE WHEN:** the evidence panel can jump from a field to the image region that produced it. **ALGORITHM:** Tesseract or PaddleOCR + field-specific parsing/regex/LLM-free normalization where possible. **CONSTRAINT:** low-confidence fields become REVIEW, not silently accepted.

M3 — Source adapters / snapshots

JOB: obtain seller listing and official declaration data. **INPUT:** product identifier/search hints. **OUTPUT:** normalized snapshots plus raw responses. **DONE WHEN:** a case can be replayed without the external service. **CONSTRAINT:** official APIs where available; policy-compliant, rate-limited acquisition otherwise; cache every demo source response.

M4 — Field normalization

JOB: make values comparable. **INPUT:** raw field strings from all three layers. **OUTPUT:** canonical values. **ALGORITHM:** currency parsing, quantity-unit conversion, date normalization, whitespace/name normalization, identifier cleanup. **DONE WHEN:** equivalent representations compare equal and genuinely different values remain different.

7. CORE VERIFICATION / RECONCILIATION ENGINE

This is the intellectual centre of TriVerify. The system should not simply ask whether one field is present; it should establish how the three independent surfaces agree or disagree.

Pairwise comparison matrix

Comparison	What it answers	Example signal
Photo ↔ Listing	Did the seller's page match what is printed on the product image?	Photo MRP 99 vs listing MRP 79
Photo ↔ Official	Does the physical label agree with the registered declaration?	Photo quantity differs from official
Listing ↔ Official	Does the online claim agree with the official declaration?	Listing manufacturer differs
All three	What pattern best explains the discrepancy?	Photo=Official, Listing differs → seller-side inconsistency

Rule result states

State	Meaning
PASS	The configured rule is satisfied by available evidence.
FAIL	The configured rule is violated by sufficiently reliable evidence.
REVIEW	Evidence exists but is missing, ambiguous, or below the configured confidence threshold.
NOT_APPLICABLE	The rule does not apply to the case/field.

Classification policy

Classification	Trigger pattern
COMPLIANT	Required checks pass and no material mismatch is found; evidence quality is sufficient.
SELLER_FRAUD	Seller/listing conflicts with photo and/or official record in a pattern attributable to the online claim.
COUNTERFEIT_OR_ILLEGAL_IMPORT	Photo/product identity conflicts materially with the official declaration, with sufficient evidence to flag the case for enforcement review.
MANUAL_REVIEW	Conflicting or incomplete evidence prevents a reliable automated classification.

Important boundary

The last classification is an engineering triage label, not a legal judgment. The source material supports evidence-backed compliance checking; it does not justify claiming that the prototype can conclusively establish criminal fraud or counterfeit status from OCR alone.

8. EVIDENCE SYSTEM — THE FEATURE THAT WINS

Every failed rule should answer three questions immediately: What was observed? Where did it come from? Why did the rule fail?

Evidence object

Property	Example
evidence_id	EV-0017
source_type	PHOTO / LISTING / OFFICIAL
source_ref	image path, listing snapshot ID, official record ID
field_name	mrp
raw_value	■99
normalized_value	99 INR
visual_ref	crop path + bounding box
captured_at	timestamp
confidence	0.96
rule_refs	MRP-001

Officer UI

- Left: product image with OCR boxes; click a field to highlight its region.
- Middle: three-column field comparison — Photo / Listing / Official.
- Right: rule findings, severity, explanation and confidence.
- A single **Evidence** action opens the raw source snapshot/crop.
- A single **Generate Report** action produces the case report.

Evidence-first design rule

Do not make the dashboard a coloured status screen. The most valuable screen is the one where a judge can click the mismatch and immediately see the underlying label/photo/source evidence. This directly follows the supplied CHIRAAG principle of evidence over unsupported claims.

Offline fallback

Pre-cache the demo product images, OCR boxes, e-commerce snapshots, official snapshots and generated report. If the network fails, the same case should still open and look identical.

9. FRONTEND / UX

One primary workflow

Avoid login, onboarding, settings and unnecessary navigation in the first demo. The officer should reach the core evidence view in one or two clicks.

Screen	Required elements
Case intake	Upload image · product identifier/source hint · Start verification
Verification result	Classification · confidence · key mismatches · rule counts
Evidence comparison	Photo/List/Official columns · clickable evidence · source timestamps
History	Case ID · product · status · date · severity · search
Report	Case summary · findings · evidence references · export

Demo hierarchy

- 1 Big classification and one-sentence reason.
- 2 The highest-severity mismatch.
- 3 The three-layer comparison table.
- 4 Click-to-evidence image crop.
- 5 Rule-by-rule details.
- 6 Report generation.

Frontend fixture strategy

At H1, the frontend should work against a hand-written JSON fixture matching the frozen API contract. The real backend should later replace the fixture without redesigning the UI.

Accessibility / clarity

- Never rely on colour alone for PASS/FAIL/REVIEW.
- Use explicit labels and confidence values.
- Show timestamps for external source snapshots.
- Keep technical rule names available but explain them in plain language.
- Make the evidence crop large enough for a judge to inspect.

10. SIX-PERSON TEAM SPLIT

Roles mirror the CHIRAAG manual's ownership philosophy: each person owns a coherent boundary, while the lead owns contracts and integration rather than becoming a bottleneck.

Role	Owns	Critical deliverables
1. Lead / Integration	Schema, API contract, integration, scope cuts	H0 contract · H6/H12/H18 merges · final demo stability
2. CV / Data	Image pipeline + OCR data flow	Preprocessing · OCR output · image fixtures
3. NLP / Normalization	Field parsing and canonical values	MRP/quantity/date/name normalization
4. Rules / Verification	Compliance rules + triangulation	Pairwise engine · classification · explanations
5. Backend / Evidence	FastAPI + DB + reports	Endpoints · evidence API · report generator
6. Frontend / UX	React dashboard	Intake · comparison · evidence viewer · history

Shared activities

- Ground-truth / sample-case preparation and edge-case review.
- Demo rehearsal.
- Rule interpretation review before claiming legal compliance.
- Final integration and failure testing.

Lead rule

After H12, the lead should spend more time unblocking, integrating and cutting scope than writing a new critical-path module.

11. EXECUTION TIMELINE

The schedule deliberately follows the supplied CHIRAAG execution model: contract first, vertical slice by H6, merge checkpoints, scope cut at H12, full demo at H18, feature freeze at H20.

Time	Target	Hard deliverable
Pre-week	Data + environment	Repo, Docker, seed data, demo images, source snapshots, initial deck
H0	Contracts	Schema + API response shape frozen
H0–H2	Skeletons	Backend routes, frontend shell, DB, upload path
H2–H6	Vertical slice	One product → OCR → normalized fields → rules → result → UI
H6	CHECKPOINT	One complete case works end-to-end
H6–H12	Core depth	Triangulation, evidence, confidence, report, real adapter wiring
H12	CHECKPOINT	Merge + DB snapshot + scope cut
H12–H18	Breadth	Multiple cases, history, polished evidence view
H18	CHECKPOINT	Full demo run with network disabled
H18–H20	Stabilize	Fix only demo-breaking defects
H20	FEATURE FREEZE	No new features
H20–H24	Demo mode	Deck, rehearsal, Q&A, final bug fixes

Checkpoint rule

At every checkpoint, merge to main, run the full smoke test, and create a recoverable snapshot. No long-lived branches and no uncommitted critical work.

12. FALLBACK MATRIX

Every module gets a degraded mode before the hackathon, not during the crisis.

Failure	Fallback	Demo impact
OCR quality poor	Use precomputed OCR fixture for demo cases; keep raw image visible	Low
E-commerce access blocked	Use cached listing snapshot; clearly label capture time	Low
Government source unavailable	Use cached official record for seeded cases; mark live lookup unavailable	Low
Product matching fails	Use explicit product identifier / manual selection in demo	Low
Normalization edge case	Route field to REVIEW and show raw values	Low
Rules engine bug	Fall back to a tested deterministic rule set + golden test cases	Medium
Report generation breaks	Render a static report fixture for the prepared demo case	Low
Frontend behind	Serve a minimal React page with core comparison/evidence only	Medium
Everything external fails	Run completely from seeded local cases	High but recoverable

Golden-case fixtures

Prepare at least four deterministic demo cases: **clean/compliant**, **seller-side mismatch**, **official-vs-photo mismatch**, and **insufficient/ambiguous evidence**. These cases become regression tests as well as the fallback demo.

13. VALIDATION & METRICS

Do not present AI confidence as legal certainty. Measure the extraction system and the rule engine separately.

Metric	How to measure	Why it matters
OCR field accuracy	Correct field/value matches on labelled demo/sample images	Shows extraction reliability
Field confidence calibration	Compare predicted confidence with observed correctness	Supports REVIEW threshold
Product/source match rate	Correctly linking photo to listing/official record	Prevents cross-product false comparisons
Rule precision	Share of FAIL findings confirmed on reviewed cases	Measures false-positive burden
Rule recall	Share of known issues detected	Measures missed violations
Manual-review rate	Fraction of cases routed to REVIEW	Shows where automation stops
End-to-end latency	Upload → result	Demo usability
Evidence completeness	Findings with usable evidence refs	Explainability

Validation protocol

- 1 Create a small labelled set of representative package images and known field values.
- 2 Run OCR/extraction and record field-level correctness.
- 3 Create known mismatch cases across the three layers.
- 4 Run the deterministic rules and compare against expected outcomes.
- 5 Inspect every false positive/negative and classify the failure reason.
- 6 Use the measured limitations in the final presentation.

Honesty statement for judges

"This prototype demonstrates the verification workflow on our prepared cases. OCR and source availability are not perfect, so low-confidence or missing evidence is explicitly routed to review rather than being treated as compliant. The legal rule set is configurable and should be validated against the authoritative current rules before production deployment."

14. REPOSITORY & TOOLING

Keep the repository obvious enough that any teammate can locate the contract, pipeline, API and frontend immediately.

```
triverify/ ■■■ data/ ■■■ raw/ # cached source responses ■■■ images/ # original + demo images ■■■
■■■ crops/ # evidence crops ■■■ fixtures/ # golden cases ■■■ pipeline/ ■■■ preprocess.py ■■■
ocr.py ■■■ extract.py ■■■ normalize.py ■■■ adapters/ ■■■ engine/ ■■■ rules.py ■■■
reconcile.py ■■■ classify.py ■■■ schemas.py ■■■ api/ ■■■ main.py ■■■ cases.py ■■■
evidence.py ■■■ reports.py ■■■ web/ ■■■ src/components/ ■■■ src/pages/ ■■■ src/fixtures/
■■■ db/ ■■■ schema.sql ■■■ seed.sql ■■■ reports/ ■■■ tests/ ■■■ test_normalization.py ■■■
test_rules.py ■■■ test_api.py ■■■ golden_cases/ ■■■ docker-compose.yml ■■■ Makefile ■■■
README.md
```

Suggested commands

Command	Purpose
make setup	Install/start local environment
make seed	Load deterministic demo cases
make test	Run unit + golden-case tests
make serve	Start FastAPI
make web	Start React
make demo	Seed + start full demo
make snapshot	Create recoverable DB/source snapshot

Version pinning

Pin Python/Node/package versions before the hackathon. The supplied CHIRAAG manual specifically treats environment reproducibility as part of the demo safety net.

15. TESTING STRATEGY

Test layer	Examples	Owner
Unit	MRP parser, quantity conversion, date normalization	NLP/Rules
Rule	Photo/listing/official mismatch combinations	Rules
Golden case	Four seeded products produce fixed classifications	Lead + Rules
API	Case create/get/verify/evidence/report	Backend
UI	Upload → result → evidence → report	Frontend
Failure	No network, missing official record, low OCR confidence	Whole team

Golden-case philosophy

The same cases used in the demo should be automated tests. This makes the demo data a safety net rather than disposable mock content.

Definition of done for P0

- Works from a clean clone using the documented commands.
- Works with network disabled for seeded cases.
- Produces deterministic results on golden cases.
- Every material finding has a rule ID and evidence reference.
- Low-confidence/missing source data produces REVIEW rather than fabricated certainty.
- Report output matches the case shown in the UI.

16. FIVE-MINUTE DEMO SCRIPT

The demo is one argument, not a feature tour.

Time	Action	What to say
0:00	Open seeded case	"This is the product we are checking. We do not trust any single source."
0:30	Show extracted fields on photo	"These values came from the physical label, with confidence and evidence."
1:10	Open listing comparison	"Now compare what the seller claims online."
1:50	Open official record	"And this is the registered declaration we use as the reference layer."
2:20	Trigger triangulation	"TriVerify checks all three pairwise and then interprets the pattern."
3:00	Click mismatch evidence	"This is the exact image region/source evidence behind the finding."
3:40	Show confidence/review	"Where evidence is weak, we do not guess — we route it to review."
4:10	Generate report	"The officer gets an evidence-backed report rather than a black-box score."
4:40	Close	"The differentiator is not OCR alone; it is the three-source reconciliation and traceable evidence."

Rehearsal rules

- Rehearse with the network disconnected.
- Have a teammate deliberately click the wrong case/button once.
- Time the demo and leave at least 20–30 seconds of buffer.
- Presenter should not be the person fixing last-minute code.
- Have the fallback seeded case openable in under 10 seconds.

17. FEATURE PRIORITIZATION

Priority	Feature	Decision
P0	Three-layer comparison	Must ship
P0	OCR field extraction + confidence	Must ship
P0	Deterministic rules engine	Must ship
P0	Evidence viewer	Must ship
P0	Classification + explanation	Must ship
P0	PDF/report export	Must ship if stable; otherwise fixture fallback
P1	Inspection history/search	Ship after core slice
P1	More product categories/rules	Only after golden cases are stable
P2	Live multi-platform crawling	Do not let it block the demo
P2	Advanced ML / retraining loop	Post-hackathon
P2	Complex RBAC/admin	Post-hackathon unless explicitly required

Scope-cut rule at H12

Protect evidence viewer → three-layer comparison → big classification/explanation → extraction confidence → report. Drop breadth before weakening the core evidence chain.

18. SECURITY, PRIVACY & RESPONSIBLE AI

- Do not store credentials in source control; use environment variables.
- Treat uploaded product images and source snapshots as potentially sensitive business data.
- Record source timestamps and identifiers so results are reproducible.
- Keep raw source payloads separate from normalized fields.
- Never claim that an OCR output is ground truth; preserve the original image.
- Use confidence thresholds and REVIEW states for ambiguous extraction.
- Avoid exposing unnecessary personal information from seller/source pages.
- For production, add authentication, authorization, encryption, retention policy and audit controls.
- Legal rules should be versioned/configurable so the engine can identify which rule set produced a result.

Responsible-AI boundary

TriVerify should be presented as a decision-support and enforcement-assistance system. It surfaces evidence and discrepancies; a qualified officer remains responsible for final enforcement action.

19. JUDGE Q&A; — READY ANSWERS

Question	Answer
Isn't this just OCR?	No. OCR is only the extraction layer. The differentiator is reconciling the physical label, seller listing and official record, then attaching evidence to the rule result.
Why not trust the official database alone?	Because the physical product and the online seller claim can diverge from the registered declaration. Triangulation reveals where the inconsistency occurs.
Why not use an LLM to decide compliance?	The supplied PS explicitly calls for rule-based compliance checking. We use AI/CV to assist extraction, while deterministic rules produce traceable findings.
What if OCR is wrong?	Each field carries confidence and visual evidence. Low-confidence fields go to REVIEW; the system does not silently convert uncertainty into compliance.
What if the e-commerce site blocks scraping?	The architecture uses adapters and cached snapshots. Live acquisition is replaceable; the verification engine operates on normalized source records.
What if the government database is incomplete?	We surface the missing source and route the case to review rather than pretending the missing record is evidence of compliance.
Can this work at scale?	The prototype separates acquisition from verification. Production scale would require robust connectors, queues, storage, rate limits and larger validation sets; the hackathon proves the core workflow.
Can this be used for legal enforcement directly?	The prototype is decision support. Production deployment would require authoritative rule validation, governance, security, auditability and human review.
What is the core innovation?	Three-source evidence reconciliation: what the product physically says, what the seller digitally says, and what the official record says.

20. FINAL IMPLEMENTATION ORDER

- 1 **Freeze the contract.** Commit DB schema, canonical fields, rule-result states and API JSON.
- 2 **Seed the demo.** Put four golden cases and all their images/source snapshots into the repo.
- 3 **Build the frontend fixture.** The UI must render the final result shape before the backend is finished.
- 4 **Build extraction.** Produce field/value/bbox/confidence objects from the image.
- 5 **Build normalization.** Make the three sources comparable.
- 6 **Build rules.** Start with a small, explicit rule set that can be explained line-by-line.
- 7 **Build triangulation.** Turn rule results into a classification and explanation.
- 8 **Build evidence.** Link every finding to source data and visual evidence.
- 9 **Wire the API.** Replace fixtures without changing the frontend contract.
- 10 **Build the report.** Make it a rendering of the structured case, not a second decision engine.
- 11 **Run the H6 vertical slice.** One product end-to-end.
- 12 **Run the H12 scope cut.** Remove anything that threatens the evidence chain.
- 13 **Run the H18 offline demo.** Disable network and find every hidden dependency.
- 14 **Feature freeze at H20.** Only bugs, rehearsal, deck and Q&A; after this point.

Final definition of success

A judge can hand the team one prepared product case and, within minutes, see the physical label evidence, the seller claim, the official declaration, the exact mismatch, the deterministic rule that fired, the confidence/uncertainty, and a generated report — even with the venue internet turned off.

21. FINAL BUILD CHECKLIST

Area	Ready?
Problem statement requirements mapped to features	■
Canonical field vocabulary frozen	■
DB schema committed	■
API response shape committed	■
Four golden cases committed	■
OCR/extraction evidence boxes working	■
Normalization tests passing	■
Deterministic rules tests passing	■
Three-layer comparison visible	■
Evidence viewer working	■
Manual-review state working	■
Report generation working or fallback ready	■
History/search working or deliberately cut	■
Network-off demo works	■
DB/source snapshot recoverable	■
Demo rehearsed 4+ times	■
Judge Q&A; prepared	■
Rule/legal wording reviewed against authoritative source	■
Feature freeze enforced	■

Source & scope note

This document is an engineering build plan based on the supplied TriVerify solution deck, the supplied SIH2026 PS034 analysis, and the supplied CHIRAAG Build Manual v1. It does not replace the authoritative Legal Metrology rules, e-Maap/CLMS documentation, or current platform/API terms. Any production legal/compliance claim must be validated against those authoritative sources.

TRIVERIFY · BUILD MANUAL V1